

# Project Assignment 3

Worcester Polytechnic Institute

Tracy Yang

Kyle Yee

July 27, 2026

## 1 Velocity-Level Kinematics (Part 1)

### 1.1 Overview

For part 1, a new node was implemented that serves two functions: taking joint velocities and converting them to end effector velocities, and taking end effector velocities and converting them to joint velocities.

### 1.2 Service Definitions

Two new services were created in the `velocity_kinematics` package.

Listing 1: JointToEEVelocity.srv

```
1 float64 theta1_dot
2 float64 theta2_dot
3 float64 d3_dot
4 ---
5 float64 vx
6 float64 vy
7 float64 vz
8 bool success
9 string message
```

Listing 2: EEToJointVelocity.srv

```
1 float64 vx
2 float64 vy
3 float64 vz
4 ---
5 float64 theta1_dot
6 float64 theta2_dot
7 float64 d3_dot
8 bool success
```

```
9 string message
```

## 1.3 Velocity Kinematics Node

Listing 3: velocity\_kinematics\_node.py

```
1 import rclpy
2 import numpy as np
3 from rclpy.node import Node
4 from sensor_msgs.msg import JointState
5 from velocity_kinematics.srv import JointToEEVelocity,
   EEToJointVelocity
6
7 A1 = 0.40
8 A2 = 0.30
9
10
11 def jacobian(theta1, theta2):
12     s1, c1 = np.sin(theta1), np.cos(theta1)
13     s12, c12 = np.sin(theta1 + theta2), np.cos(theta1 + theta2)
14     return np.array([
15         [-A1*s1 - A2*s12, -A2*s12, 0],
16         [ A1*c1 + A2*c12,  A2*c12, 0],
17         [ 0,              0,      1]
18     ])
19
20
21 class VelocityKinematicsNode(Node):
22
23     def __init__(self):
24         super().__init__('velocity_kinematics_node')
25         self.theta1 = 0.0
26         self.theta2 = 0.0
27
28         self.create_subscription(JointState, '/joint_states', self.
            joint_state_cb, 10)
29         self.create_service(JointToEEVelocity, '/scara/
            joint_to_ee_velocity', self.fwd_vel_cb)
30         self.create_service(EEToJointVelocity, '/scara/
            ee_to_joint_velocity', self.inv_vel_cb)
31
32     def joint_state_cb(self, msg):
33         if 'joint_1' in msg.name:
34             self.theta1 = msg.position[msg.name.index('joint_1')]
35         if 'joint_2' in msg.name:
36             self.theta2 = msg.position[msg.name.index('joint_2')]
37
```

```

38     def fwd_vel_cb(self, request, response):
39         qdot = [request.theta1_dot, request.theta2_dot, request.
40                 d3_dot]
41         vx, vy, vz = jacobian(self.theta1, self.theta2) @ qdot
42         response.vx, response.vy, response.vz = vx, vy, vz
43         response.success = True
44         return response
45
46     def inv_vel_cb(self, request, response):
47         xdot = [request.vx, request.vy, request.vz]
48         Jv = jacobian(self.theta1, self.theta2)
49         try:
50             q1, q2, q3 = np.linalg.solve(Jv, xdot)
51         except np.linalg.LinAlgError:
52             response.theta1_dot, response.theta2_dot, response.
53                 d3_dot = 0.0, 0.0, 0.0
54             response.success = False
55             response.message = (
56                 f'Singular Jacobian at theta1={self.theta1:.4f},
57                 theta2={self.theta2:.4f} '
58                 '(theta2 near 0 or pi) -- no unique joint-velocity
59                 solution.'
60             )
61             self.get_logger().warn(response.message)
62             return response
63         response.theta1_dot, response.theta2_dot, response.d3_dot =
64             q1, q2, q3
65         response.success = True
66         return response
67
68     def main(args=None):
69         rclpy.init(args=args)
70         node = VelocityKinematicsNode()
71         rclpy.spin(node)
72         node.destroy_node()
73         rclpy.shutdown()
74
75 if __name__ == '__main__':
76     main()

```

In the velocity kinematics node, there are two main functions: forward velocity callback and inverse velocity callback. These functions take theta 1 and theta 2 from a joint state subscriber and calculate the jacobian using a separate function. With the jacobian, the forward velocity function multiplies it with qdot to get vx, vy, and vz. The inverse velocity function solves for qdot in the same equation. This equation can fail if the determinant of Jv is 0

(at  $\theta_2 = 0$  or  $\pi$ , the arm’s fully-extended or fully-folded configuration), so `np.linalg.solve` is wrapped in a `try/except` on `LinAlgError`: on a singular Jacobian the service returns `success = false` with a diagnostic message instead of letting the exception propagate and crash the node. This matters beyond Part 1 itself – Part 4’s constant-velocity node (Section 4) calls this service every cycle and depends on it failing gracefully rather than taking the whole service down.

## 2 Extending the Position Controller (Part 2)

### 2.1 Overview

In part 2, the URDF and controller node were updated to extend the position controller from part 2.

### 2.2 URDF Changes

The joint types for joints 1 and 2 were changed back from fixed to revolute. The Gazebo actuation plugin block was then updated so all three joints are driven by a real effort input.

Listing 4: joint\_1 and joint\_2 reverted to revolute

```

1 <joint name="joint_1" type="revolute">
2   <parent link="base_link"/><child link="link_1"/>
3   <origin xyz="0 0 0.5"/><axis xyz="0 0 1"/>
4   <limit effort="1000" lower="-1.57" upper="1.57" velocity="0.5"/>
5 </joint>
6
7 <joint name="joint_2" type="revolute">
8   <parent link="arm_1"/><child link="link_2"/>
9   <origin xyz="0.4 0 0"/><axis xyz="0 0 1"/>
10  <limit effort="1000" lower="-1.57" upper="1.57" velocity="0.5"/>
11  <dynamics damping="0.2" friction="0.1"/>
12 </joint>

```

**Actuation plugin debugging.** The first attempt used Gazebo’s `JointPositionController` plugin per joint, with the controller node publishing its computed PD effort onto that plugin’s command topic. In simulation the joints simply refused to move: the commanded values (tens of N·m) were being interpreted as absolute *position* targets rather than effort, and since those targets fell far outside each joint’s  $\pm 1.57$  rad limit the plugin applied no torque at all. Setting `use_force_commands` on that plugin did not change this behavior. The fix was to drop `JointPositionController` entirely and drive every joint through a single `JointController` plugin per joint in force mode – the same plugin type used for Part 3 – confirmed by publishing a raw effort value directly to its command topic and observing the joint move exactly as expected. Both the position controller (Part 2) and the velocity controller (Part 3) now publish onto the same `/scara/jointX_effort_cmd` topic per joint;

only one of the two ROS nodes is ever run at a time; running both simultaneously would have them fight over which effort value the single plugin instance actually applies.

Listing 5: Single force-mode actuation plugin per joint

```

1 <gazebo>
2   <plugin filename="gz-sim-joint-state-publisher-system"
3     name="gz::sim::systems::JointStatePublisher">
4     <topic>/world/empty/model/scara/joint_state</topic>
5     <joint_name>joint_1</joint_name>
6     <joint_name>joint_2</joint_name>
7     <joint_name>joint_3</joint_name>
8   </plugin>
9
10  <plugin filename="gz-sim-joint-controller-system"
11    name="gz::sim::systems::JointController">
12    <joint_name>joint_1</joint_name>
13    <use_force_commands>true</use_force_commands>
14    <topic>/scara/joint1_effort_cmd</topic>
15    <p_gain>1</p_gain><i_gain>0</i_gain>
16  </plugin>
17  <plugin filename="gz-sim-joint-controller-system"
18    name="gz::sim::systems::JointController">
19    <joint_name>joint_2</joint_name>
20    <use_force_commands>true</use_force_commands>
21    <topic>/scara/joint2_effort_cmd</topic>
22    <p_gain>1</p_gain><i_gain>0</i_gain>
23  </plugin>
24  <plugin filename="gz-sim-joint-controller-system"
25    name="gz::sim::systems::JointController">
26    <joint_name>joint_3</joint_name>
27    <use_force_commands>true</use_force_commands>
28    <topic>/scara/joint3_effort_cmd</topic>
29    <p_gain>1</p_gain><i_gain>0</i_gain>
30  </plugin>
31 </gazebo>

```

## 2.3 Controller Node Changes

To reuse the same position controller from part 2, it had to be adjusted to work for multiple joints. This was done by creating dictionaries for each of the stored values. Now, each of the variables that were originally just for a singular value have values for each of the joints.

Listing 6: Per-joint state, publishers, and services

```

1 JOINT_NAMES = ['joint_1', 'joint_2', 'joint_3']
2
3 KP = {'joint_1': 50.0, 'joint_2': 50.0, 'joint_3': 800.0}
4 KD = {'joint_1': 5.0, 'joint_2': 5.0, 'joint_3': 80.0}

```

```

5 JOINT_MIN = {'joint_1': -1.57, 'joint_2': -1.57, 'joint_3': -0.20}
6 JOINT_MAX = {'joint_1': 1.57, 'joint_2': 1.57, 'joint_3': 0.20}
7
8 self.ref_position = {j: 0.0 for j in JOINT_NAMES}
9 self.cur_position = {j: 0.0 for j in JOINT_NAMES}
10 self.prev_error = {j: 0.0 for j in JOINT_NAMES}
11
12 self.cmd_pub = {
13     j: self.create_publisher(Float64, f'/scara/{j.replace("_","")}
14         _effort_cmd', 10)
15     for j in JOINT_NAMES
16 }
17
18 self.srv = {
19     j: self.create_service(
20         SetJointPosition,
21         f'/scara/set_{j.replace("_","")} _position',
22         self.make_set_position_cb(j)
23     )
24     for j in JOINT_NAMES
25 }

```

Listing 7: Callback factory — binds a joint name to its own service callback

```

1 def make_set_position_cb(self, joint_name):
2     def set_position_cb(request, response):
3         target = request.position
4         if not (JOINT_MIN[joint_name] <= target <= JOINT_MAX[
5             joint_name]):
6             response.success = False
7             response.message = f'Requested {target:.4f} outside
8                 limits for {joint_name}.'
9             return response
10        self.ref_position[joint_name] = target
11        response.success = True
12        response.message = f'{joint_name} reference set to {target
13            :.4f}'
14        return response
15    return set_position_cb

```

Listing 8: PD control loop, generalized to all three joints

```

1 def control_loop(self):
2     now = self.get_clock().now()
3     if self.prev_time is None:
4         self.prev_time = now
5         return
6     dt = (now - self.prev_time).nanoseconds * 1e-9
7     if dt <= 0.0:

```

```

8         return
9     self.prev_time = now
10
11     for j in JOINT_NAMES:
12         error = self.ref_position[j] - self.cur_position[j]
13         error_dot = (error - self.prev_error[j]) / dt
14         self.prev_error[j] = error
15
16         effort = KP[j] * error + KD[j] * error_dot
17         effort = max(-EFFORT_LIMIT, min(EFFORT_LIMIT, effort))
18
19         msg = Float64()
20         msg.data = effort
21         self.cmd_pub[j].publish(msg)

```

## 3 Velocity Controllers (Part 3)

### 3.1 Overview

Each joint gets a PI velocity controller rather than reusing the Part 2 PD law:

$$\tau = K_p e_v + K_i \int e_v dt, \quad e_v = \dot{q}_{ref} - \dot{q}_{cur}.$$

Differentiating the velocity signal (as PD would) amplifies measurement noise; the integral term instead removes steady-state error without a derivative term.

The controller subscribes to `/joint_states` for measured velocity and to per-joint `/scara/jointX_vel_ref` topics for the reference, publishing effort on the same `/scara/jointX_effort_cmd` topics used in Part 2 (only one of the two ROS controllers runs at a time).

### 3.2 Velocity Controller Node

Listing 9: PI gains and per-joint state/topics

```

1 JOINT_NAMES = ['joint_1', 'joint_2', 'joint_3']
2
3 KP = {'joint_1': 20.0, 'joint_2': 0.5, 'joint_3': 8.0}
4 KI = {'joint_1': 2.0, 'joint_2': 0.4, 'joint_3': 1.0}
5
6 self.ref_velocity = {j: 0.0 for j in JOINT_NAMES}
7 self.cur_velocity = {j: 0.0 for j in JOINT_NAMES}
8 self.integral      = {j: 0.0 for j in JOINT_NAMES}
9
10 self.ref_subs = {
11     j: self.create_subscription(

```

```

12         Float64, f'/scara/{j.replace("_", "")}_vel_ref', self.
           make_ref_cb(j), 10)
13     for j in JOINT_NAMES
14 }
15 self.cmd_pub = {
16     j: self.create_publisher(Float64, f'/scara/{j.replace("_", "")}_
       _effort_cmd', 10)
17     for j in JOINT_NAMES
18 }

```

Listing 10: PI control loop

```

1 for j in JOINT_NAMES:
2     error = self.ref_velocity[j] - self.cur_velocity[j]
3
4     self.integral[j] += error * dt
5     self.integral[j] = max(-INTEGRAL_LIMIT, min(INTEGRAL_LIMIT, self
       .integral[j]))
6
7     effort = KP[j] * error + KI[j] * self.integral[j]
8     effort = max(-EFFORT_LIMIT, min(EFFORT_LIMIT, effort))
9
10    msg = Float64()
11    msg.data = effort
12    self.cmd_pub[j].publish(msg)

```

The integral term is clamped (anti-windup) so a large or long-lived error – e.g. while a joint accelerates toward a step reference – cannot saturate the integrator and cause overshoot once the error shrinks.

### 3.3 Gain Tuning

Gains were first tuned per joint in isolation, temporarily setting the other two joints' URDF types to `fixed` (reverted afterward). Equal gains ( $K_p=20$ ,  $K_i=2$ ) worked for joint\_1 and joint\_2 alone; joint\_3 needed a much larger pair ( $K_p=400$ ,  $K_i=40$ ).

Running all three joints together for Part 4 exposed a problem isolated tuning missed: joint\_2 and joint\_3 chattered against their  $\pm 0.5$  rad/s limits instead of settling, while joint\_1 tracked smoothly. The cause is inertia mismatch – link\_2's  $I_{zz}=0.01$  is  $\sim 50\times$  smaller than link\_1's ( $I_{zz}=0.5$ ), and link\_3 is only 0.1 kg – so the same effort gain that gives joint\_1 a modest, damped acceleration gives joint\_2/3 a huge one, overshooting the limit and oscillating against it. Scaling gains down roughly with each joint's effective inertia (to  $K_p=0.5$ ,  $K_i=0.4$  for joint\_2) killed the chatter.

That fix went too far, though. With chatter gone, Figure 1 still showed joint\_2's measured velocity visibly lagging its reference by a growing gap that never closed within the test window, while joint\_1 and joint\_3 tracked cleanly. The difference is what each joint is being



asked to track: joint\_1 and joint\_3's Part 4 references are nearly constant, so the step-response-style gains from isolated tuning still apply, but joint\_2's reference is a continuously-decreasing ramp – holding a constant  $+y$  Cartesian velocity requires  $\theta_2$  to keep shrinking (see Part 4) – and gains damped enough to avoid step-input chatter were too sluggish to keep up with a moving target. joint\_2's gains were raised back up from that low point until the lag closed, without reintroducing the original chattering:

|       | joint_1 | joint_2 | joint_3 |
|-------|---------|---------|---------|
| $K_p$ | 20.0    | 3.0     | 8.0     |
| $K_i$ | 2.0     | 1.5     | 1.0     |

Lesson: gains tuned on a single isolated joint against a step input aren't guaranteed stable, or even sufficient, once the full coupled system runs together against a reference that keeps moving – both need to be re-checked against the actual Part 4 conditions, not just the isolated step-response test from Part 3.

## 4 Constant Velocity Reference (Part 4)

### 4.1 Overview

Part 4 drives the end effector in a straight  $+y$  line by holding a constant Cartesian velocity  $(\dot{x}, \dot{y}, \dot{z}) = (0, v_y, 0)$  and converting it to joint rates via the Part 1 inverse velocity service. Since the Jacobian depends on the current configuration, this is re-solved on a timer as the robot moves rather than solved once.

Listing 11: constant\_velocity\_node.py — Part 4

```

1 JOINT_NAMES = ['joint_1', 'joint_2', 'joint_3']
2 VY = 0.05 # m/s, constant end-effector velocity in +y
3
4 class ConstantVelocityNode(Node):
5
6     def __init__(self):
7         super().__init__('constant_velocity_node')
8         self.cli = self.create_client(EEToJointVelocity, '/scara/
          ee_to_joint_velocity')
9         self.ref_pub = {
10             j: self.create_publisher(Float64, f'/scara/{j.replace("_",
              "")}_vel_ref', 10)
11             for j in JOINT_NAMES
12         }
13         self.pending = False
14         self.timer = self.create_timer(0.05, self.timer_cb) # 20 Hz
15
16     def timer_cb(self):
17         if self.pending:
18             return

```

```

19     req = EEToJointVelocity.Request()
20     req.vx, req.vy, req.vz = 0.0, VY, 0.0
21     self.pending = True
22     future = self.cli.call_async(req)
23     future.add_done_callback(self.response_cb)
24
25     def response_cb(self, future):
26         self.pending = False
27         resp = future.result()
28         if not resp.success:
29             self.get_logger().warn('Inverse velocity solve failed (
30                                     near singularity?)')
31             return
32         qdot = {'joint_1': resp.theta1_dot, 'joint_2': resp.
33                 theta2_dot, 'joint_3': resp.d3_dot}
34         for j in JOINT_NAMES:
35             self.ref_pub[j].publish(Float64(data=qdot[j]))

```

The service call is asynchronous (`call_async` with a `done_callback`) since a blocking wait inside the same executor would deadlock the node against its own response.

Two failure modes are handled. First,  $\det(J_v) = a_1 a_2 \sin \theta_2 = 0$  at  $\theta_2=0$ : Part 1 now returns `success = false` instead of raising a linear-algebra exception, and this node just skips that cycle's reference rather than crashing – so the robot must first be moved off  $\theta_2=0$  using the Part 2 position controller. Second, a constant  $+y$  velocity can itself drive  $\theta_2$  back toward zero over time, so required joint rates grow the longer the reference is held and the arm can hit joint limits within seconds. The results below use a short window from a start pose well clear of  $\theta_2=0$ ; a robust implementation would detect this drift and stop or replan before saturating.

## 4.2 Logging and Plotting

The Part 3 velocity controller logs reference and measured velocity for all three joints to `velocity_log.csv` at each 100 Hz control cycle, in the same format as the Part 2 position log.

Listing 12: `plot_velocity.m`

```

1 data = readmatrix(fullfile(script_dir, 'velocity_log.csv'));
2 t = data(:,1);
3 for i = 1:3
4     ref = data(:, 2*i);
5     cur = data(:, 2*i + 1);
6     subplot(3,1,i);
7     plot(t, ref, 'r--', 'LineWidth', 1.5); hold on;
8     plot(t, cur, 'b-', 'LineWidth', 1.5);
9     grid on; xlabel('Time (s)'); legend('Reference', 'Measured');
10 end

```

### 4.3 Results

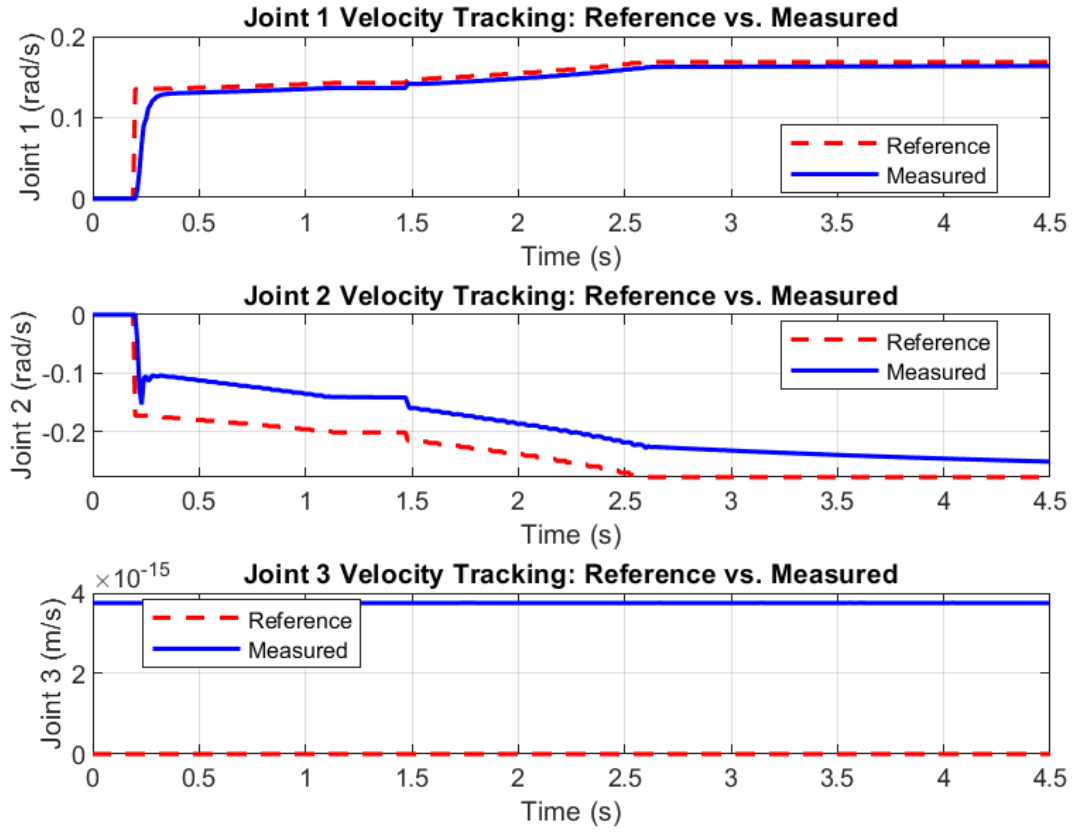


Figure 1: Reference vs. measured joint velocity for all three joints while tracking a constant  $+y$  end-effector velocity of  $v_y = 0.05$  m/s.